Aprendizaje por Refuerzo.

Michele Sechi
michele.sechi@um.es

Giovanni Corona
giovannibattista.corona@um.es

15 de marzo de 2026

Resumen

En este documento se presentará uno de los trabajos que puede realizar como proyecto final y que deberán de ser expuestos en público.

Recuerde que la experimentación deberá de estar en un repositorio GitHub para poder contrastar lo que aquí explique. En el GitHub puede añadir tantos *notebooks* como considere. Pero aquí, el trabajo documental, ocupará 10 páginas descontadas la portada, el resumen, el índice y la bibliografía.

El contenido del repositorio y este documento deberán subirse al AV como evidencias del trabajo realizado.

Respete las dobles columnas. Si las suprime generará un documento que ocupará más páginas. Con las dobles columnas puede incluir más contenido.

Índice general

1. Problema del Bandido de k-brazos **3**

1.1. Introducción . . . 3

1.2. Desarrollo . . . 3

1.3. Algoritmos . . . 3

1.4. Evaluación y Experimentos . . . 3

1.4.1. Configuración Experimental . . . 3

1.4.2. Entorno Binomial . . . 4

1.4.3. Entorno Bernoulli . . . 4

1.4.4. Entorno Normal: Resistencia a la Varianza . . . 4

1.4.5. Análisis Intrafamilia UCB . . . 5

1.5. Conclusiones . . . 5

2. Aprendizaje en Entornos Complejos **6**

2.1. Métodos Tabulares . . . 6

2.1.1. Objetivos . . . 6

2.1.2. Desarrollo. . . . 6

2.1.3. Algoritmos. . . . 6

2.2. Evaluación / Experimentos . . . 6

2.2.1. Configuración experimental . . . 7

2.2.2. Métricas utilizadas. . . . 7

2.2.3. Resultados obtenidos. . . . 7

2.2.4. Análisis . . . 7

2.3. Conclusiones . . . 8

2.4. Métodos Aproximados. . . . 8

2.4.1. Objetivos . . . 9

2.4.2. Desarollo. . . . 9

2.4.3. Algoritmos. . . . 9

2.5. Evaluación / Experimentos 9
2.5.1. Configuración experimental. 9
2.5.2. Métricas utilizadas. 10
2.5.3. Resultados obtenidos. 10
2.5.4. Análisis. 11
2.5.5. Análisis de sensibilidad. 12
2.6. Conclusiones. 12

Capítulo 1

Problema del Bandido de k-brazos

1.1. Introducción

- **Descripción del problema:** El problema del bandido multibrazo (k-armed bandit) modela una situación fundamental de toma de decisiones secuencial bajo incertidumbre. En cada instante de tiempo t, un agente debe elegir una acción a_t entre k opciones disponibles (brazos), recibiendo una recompensa r_t extraída de una distribución de probabilidad desconocida $\mathcal{P}_{a_t}$.
- **Motivación:** La relevancia de este problema radica en que encapsula el dilema central del Aprendizaje por Refuerzo: la tensión entre la *exploración* (probar acciones subóptimas para descubrir su verdadero valor) y la *explotación* (seleccionar la acción con el mayor valor esperado conocido).
- **Objetivos:** El propósito de este estudio es evaluar empíricamente la eficiencia de diversas familias algorítmicas frente a topologías de recompensa de distinta naturaleza (binaria, discreta y continua con alto ruido estocástico).
- **Organización:** El documento define formalmente los entornos evaluados, describe los algoritmos implementados y, finalmente, expone un análisis crítico de los resultados experimentales centrado en la convergencia y la acumulación del *regret* (rechazo).

1.2. Desarrollo

Para garantizar la robustez del estudio empírico, se han diseñado tres entornos estocásticos distintos, cada uno modelado bajo una distribución de probabilidad específica para simular diferentes escenarios del mundo real.

- **Distribución Bernoulli:** Modela entornos de recompensa binaria (éxito o fracaso, $r_t \in \{0, 1\}$). Es ampliamente utilizado en la optimización de clics publicitarios (CTR).
- **Distribución Binomial ($\mathcal{B}(n, p)$):** Evalúa el rendimiento sobre lotes de eventos independientes. En nuestro diseño, simula la asignación de promociones a lotes de $n = 50$ usuarios, donde la recompensa es el número de conversiones logradas.
- **Distribución Normal ($\mathcal{N}(\mu, \sigma^2)$):** Introduce el desafío de las métricas continuas con varianza paramétrica. Representa escenarios de alta volatilidad, como el tiempo de visualización en sistemas de recomendación, donde el ruido (σ) puede enmascarar la verdadera media de la recompensa (μ).

El objetivo matemático subyacente de todos los agentes es minimizar el *regret* acumulado $R(T)$, definido como la pérdida esperada por no haber seleccionado siempre la acción óptima μ^*:

$$R(T) = T\mu^* - \mathbb{E}\left[\sum_{t=1}^{T} \mu_{a_t}\right] \tag{1.1}$$

1.3. Algoritmos

Para el estudio comparativo se han implementado las siguientes familias algorítmicas, cada una con un enfoque heurístico o probabilístico distinto para abordar la exploración:

- **Familia Greedy (ϵ-Greedy y ϵ-Decay):** Basan sus decisiones puramente en el valor esperado $\hat{Q}_t(a)$. Con probabilidad $1 - \epsilon$ explotan el mejor brazo conocido, y con probabilidad ϵ exploran uniformemente. La variante *Decay* reduce agresivamente ϵ a lo largo del tiempo para forzar la convergencia asintótica.
- **Familia UCB (Upper Confidence Bound):** Aplican el principio de "optimismo frente a la incertidumbre". UCB1 suma a la estimación empírica un término de exploración $c\sqrt{\frac{\ln t}{N_t(a)}}$ que crece para los brazos menos muestreados. La variante UCB2 agrupa el aprendizaje en épocas de longitud exponencial controladas por un parámetro α, reduciendo la frecuencia de actualización.
- **Métodos Basados en Probabilidad (Softmax y Gradient Bandit):** En lugar de decisiones deterministas, utilizan distribuciones de probabilidad. Softmax convierte los valores Q en probabilidades mediante la distribución de Boltzmann (gobernada por una temperatura τ). Gradient Bandit no estima recompensas en absoluto, sino que actualiza "preferencias" relativas ($H_t(a)$) mediante el ascenso del gradiente estocástico frente a una línea base (baseline) de recompensa media.

1.4. Evaluación y Experimentos

1.4.1. Configuración Experimental

Para asegurar la validez de los resultados, los experimentos se han ejecutado promediando 500 simulaciones Monte Carlo independientes. El horizonte temporal se ha fijado entre 5000 y 6000 pasos, garantizando la observación del comportamiento asintótico.

Las topologías de recompensa de cada entorno han sido diseñadas manualmente con el propósito de someter a los al-

goritmos a condiciones analíticamente exigentes. En el entorno Bernoulli ($k = 11$) se ha configurado un brazo óptimo ($p = 0{,}20$) rodeado de múltiples competidores con probabilidades escalonadas y próximas entre sí, forzando a los algoritmos a discriminar diferencias reducidas en un espacio de acciones amplio.En el entorno Binomial ($k = 5$, $n = 50$) se han definido brazos con probabilidades de conversión próximas, evaluando la robustez ante el ruido de los lotes. En el entorno Normal ($k = 10$) se han asignado desviaciones estándar heterogéneas ($\sigma \in [0{,}5, 5{,}0]$), otorgando deliberadamente la mayor volatilidad al brazo óptimo ($\sigma = 5{,}0$) para analizar el comportamiento de los algoritmos cuando el ruido del mejor brazo enmascara su verdadera superioridad frente a competidores más estables.

1.4.2. Entorno Binomial

El objetivo fue identificar el brazo óptimo (Índice 2) que ofrecía una recompensa esperada de 9.0 sobre lotes de $n = 50$.

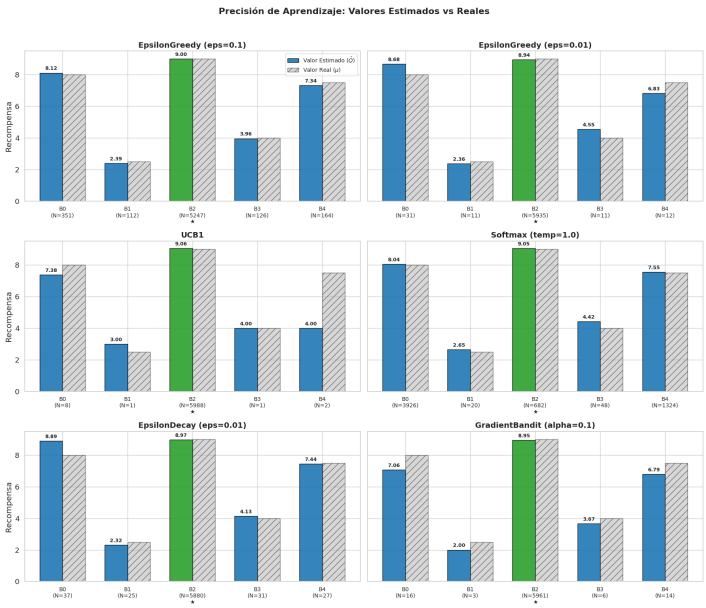

Figura 1.1: Valores Q Estimados vs Reales (Binomial). Se ilustra la convergencia en el brazo óptimo (verde) y las desviaciones exploratorias en brazos subóptimos.

Como se observa en la Figura 1.1, algoritmos como UCB1 convergen con alta precisión al valor real del brazo óptimo ($\hat{Q} = 9{,}06$ frente a $\mu = 9{,}0$), concentrando la práctica totalidad de sus selecciones en dicho brazo ($N = 5988$). El hallazgo analítico más relevante reside en la brecha entre la estimación y el valor real de los brazos subóptimos. Esto no constituye un defecto de cálculo, sino la prueba de una exploración eficiente: el algoritmo "congela" tempranamente las estimaciones de los peores brazos al adquirir confianza estadística, minimizando el *regret*.

Es crucial destacar el fracaso crítico de **Softmax** ($\tau = 1{,}0$). Como evidencia su distribución de selecciones ($N = 3926$ en B0, $N = 682$ en el brazo óptimo B2, $N = 1324$ en B4), el algoritmo dispersa sus muestras de forma casi uniforme entre todos los brazos. Este resultado demuestra su extrema sensibilidad a la temperatura: al emplear la función exponencial, si τ no está calibrada milimétricamente para la magnitud de las recompensas, el sistema colapsa. Una temperatura $\tau = 1{,}0$ aplastó las diferencias entre los valores Q, forzando una exploración casi uniforme e impidiendo la identificación del óptimo.

1.4.3. Entorno Bernoulli

El entorno evaluó un espacio de 11 acciones donde el mejor brazo poseía una probabilidad de éxito $p = 0{,}2$.

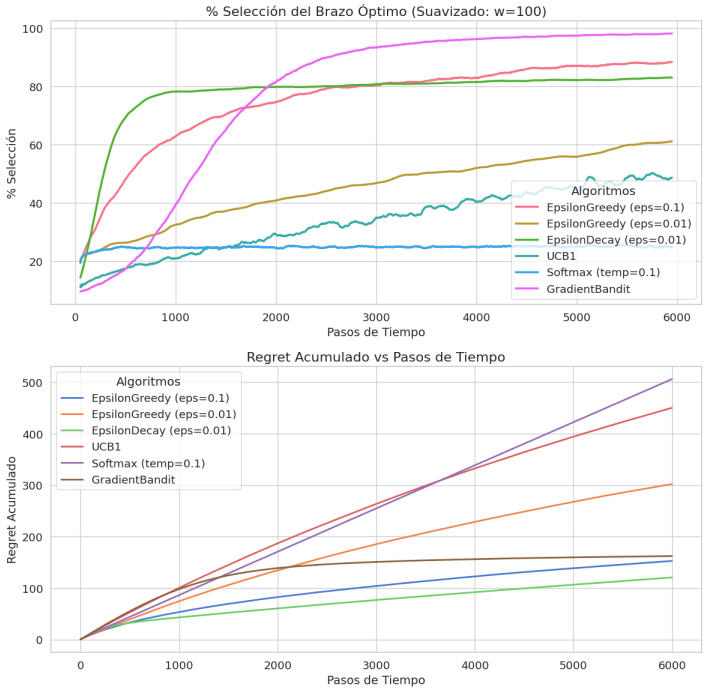

Figura 1.2: Izquierda: % de selección del brazo óptimo. Derecha: Regret Acumulado (Bernoulli).

La Figura 1.2 muestra que *Gradient Bandit* domina la tasa de selección porcentual, aislando el brazo óptimo hasta alcanzar un $\approx 98\,\%$ de selección al final del horizonte. Sin embargo, al cruzar esta métrica con el *Regret*, se evidencia que **ϵ-Decay** es el algoritmo con el menor coste acumulado de aprendizaje (≈ 120), seguido a distancia por UCB1 y Gradient Bandit, que alcanzan niveles de *regret* comparables entre sí (≈ 160). La ventaja de ϵ-Decay radica en su transición rápida de exploración a explotación, favorecida por un gap $\Delta = 0{,}07$ suficiente para ser detectado antes del agotamiento de su presupuesto exploratorio. Gradient Bandit, por su parte, requiere una fase inicial de exploración agresiva para estabilizar su *baseline* global de recompensa media, lo cual penaliza significativamente el *regret* temprano. Cabe señalar el rendimiento deficiente de Softmax ($\tau = 0{,}1$) y ϵ-Greedy ($\epsilon = 0{,}01$), cuya exploración insuficiente los condena a explotar brazos subóptimos durante la mayor parte del horizonte temporal, acumulando el mayor *regret* del estudio (≈ 500).

1.4.4. Entorno Normal: Resistencia a la Varianza

El escenario Normal ($k = 10$) introduce varianza extrema. El brazo óptimo presentaba una media $\mu = 15{,}90$ con una desviación estándar muy amplia $\sigma = 5{,}0$, generando recompensas fuertemente ruidosas.

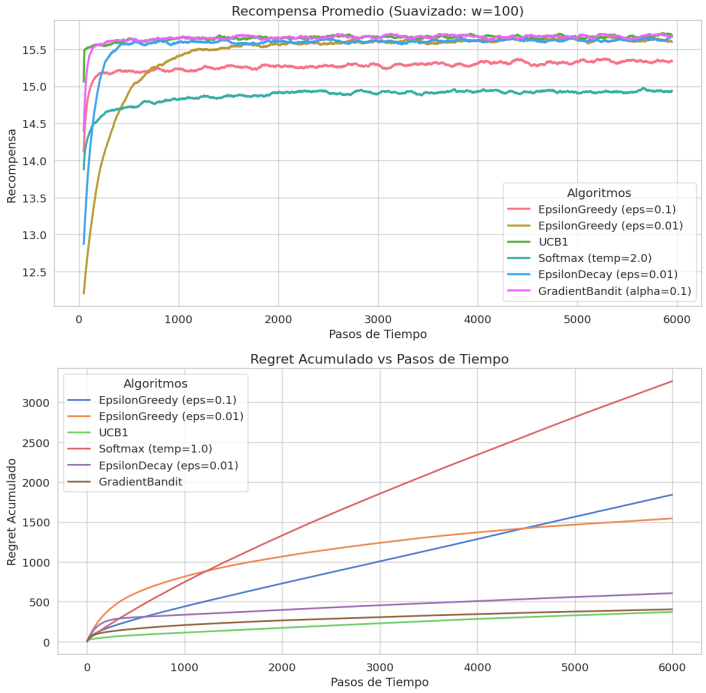

Figura 1.3: Recompensa Promedio y Regret Acumulado (Normal de alta varianza).

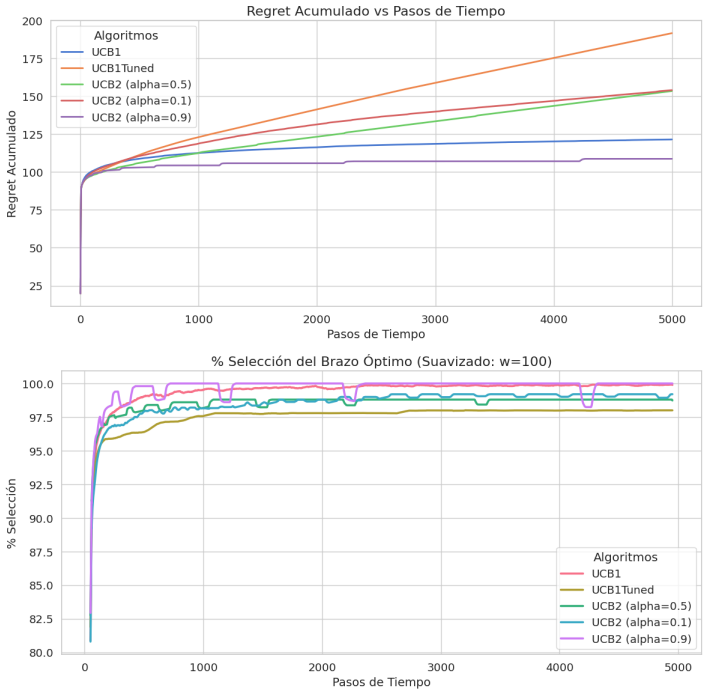

Figura 1.4: Comparativa del Regret y Selección Óptima para la familia UCB ($\sigma = 1$).

La Figura 1.3 expone el límite de las heurísticas puras bajo condiciones de alta volatilidad. Softmax ($\tau = 1{,}0$) acumula el mayor coste de *regret*, con un crecimiento marcadamente lineal que refleja una convergencia prematura hacia un brazo subóptimo. De modo similar, ϵ-Greedy con $\epsilon = 0{,}1$ y $\epsilon = 0{,}01$ mantienen trayectorias lineales de *regret*, aunque a escalas distintas: la variante con mayor exploración ($\epsilon = 0{,}1$) logra finalmente una recompensa promedio superior, pero su tasa de exploración constante le impide reducir el coste acumulado a largo plazo. En contraste, **UCB1** absorbe la volatilidad de manera sobresaliente; su optimismo intrínseco le obliga a seguir explorando los brazos ruidosos hasta que el colapso de los márgenes de confianza revela la verdadera media subyacente, alcanzando una de las recompensas promedio más altas ($\approx 15{,}7$) junto con ϵ-Decay y Gradient Bandit.

1.4.5. Análisis Intrafamilia UCB

El último análisis evalúa la familia UCB en un entorno con bajo ruido ($\sigma = 1$ para todos los brazos).

Al existir poca superposición entre distribuciones, las diferencias de diseño dentro de la familia UCB quedan al descubierto. **UCB1** muestra un rendimiento sólido con un *regret* final competitivo (≈ 120), aunque su diseño iterativo — al recalcular los límites de confianza en cada paso — lo obliga a revisitar periódicamente brazos subóptimos, impidiéndole alcanzar el rendimiento óptimo absoluto. Resulta revelador el comportamiento de **UCB1-Tuned** (curva naranja): diseñado para aprovechar la varianza empírica, su estimación inicial introduce un margen de incertidumbre ficticio ante la escasez de ruido real, forzándolo a sobre-explorar inútilmente y relegándolo a un *regret* cercano a ≈ 155, el peor de las variantes junto con UCB2 ($\alpha = 0{,}5$). En el extremo opuesto, **UCB2 ($\alpha = 0{,}9$)** agrupa el aprendizaje en épocas de crecimiento exponencial. Al calibrar $\alpha = 0{,}9$, el algoritmo logra aislar el óptimo de inmediato y "bloquearlo", extendiendo al máximo las fases de explotación ininterrumpida, alcanzando un porcentaje de selección del brazo óptimo cercano al 100 % antes que cualquier otra variante y marcando el límite de rendimiento absoluto del experimento con un *regret* mínimo de ≈ 110.[1]

1.5. Conclusiones

Resumen Analítico: El estudio demuestra que no existe un algoritmo universalmente superior, sino dependencias topológicas severas entre el rendimiento y la naturaleza del entorno.[2] Mientras UCB1 ofrece las garantías asintóticas más sólidas bajo alta incertidumbre y varianza, optimizaciones temporales como las épocas exponenciales de UCB2 ($\alpha = 0{,}9$) dominan en entornos de bajo ruido al suprimir la tasa constante de exploración. En entornos con recompensas discretas de baja magnitud (Bernoulli), ϵ-Decay destaca por su transición eficiente hacia la explotación pura. El análisis de escalabilidad confirma además que los métodos con exploración adaptativa (UCB1) mantienen un rendimiento viable a medida que crece el espacio de acciones, frente al deterioro acelerado de las políticas con exploración uniforme. **Análisis Transversal:** UCB1 es el único algoritmo que mantiene un rendimiento competitivo en las tres distribuciones sin ajuste de hiperparámetros. Softmax exhibe la mayor fragilidad, colapsando ante calibraciones inadecuadas de τ en todos los entornos. Gradient Bandit muestra robustez ante cambios de escala al operar sobre preferencias relativas, mientras que ϵ-Decay resulta eficiente en horizontes finitos solo cuando el gap Δ es suficiente para evitar la convergencia prematura. **Limitaciones:** Softmax y UCB1-Tuned evidencian una extrema fragilidad ante la calibración inadecuada de sus hiperparámetros (τ, α).

Capítulo 2

Aprendizaje en Entornos Complejos

En esta segunda parte del trabajo nos centramos en entornos complejos modelados como procesos de decisión de Markov, donde el espacio de estados y acciones puede ser discreto y finito (métodos tabulares) o continuo y de alta dimensionalidad (métodos aproximados).

El objetivo general es doble: por un lado, implementar y comparar los principales algoritmos tabulares en un entorno de cuadrícula; por otro, ampliar la comparación a los métodos con aproximación de funciones en un entorno con estados continuos.

2.1. Métodos Tabulares

2.1.1. Objetivos

1. Implementar y comparar cuatro algoritmos tabulares: Q-Learning, SARSA, Monte Carlo On-Policy y Monte Carlo Off-Policy.

2. Analizar la velocidad de convergencia, la cobertura del espacio de estados y la optimización de las trayectorias aprendidas.

3. Identificar los puntos fuertes y las limitaciones de cada enfoque en un entorno controlado.

2.1.2. Desarrollo.

- **Definición formal:** el problema se modela como un Proceso de Decisión de Markov finito, donde el objetivo es maximizar la recompensa acumulada en un entorno con espacios de estados S y de acciones A discretos y acotados. La función de valor se representa mediante una tabla Q.
- **Métodos utilizados:** Se implementaron y compararon cuatro enfoques: Monte Carlo on-policy y off-policy, SARSA como método on-policy y Q-Learning como método off-policy.

2.1.3. Algoritmos.

Q-Learning (Off-Policy TD)

El aprendizaje Q es un método de control TD Off Policy. La función Q aproxima directamente q^*, independientemente de la política de comportamiento, utilizando la ecuación óptima de Bellman:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) \\ &\quad + \alpha\big[R_{t+1} + \gamma \max_{a'} Q(S_{t+1}, a') - Q(S_t, A_t)\big] \end{aligned} \tag{2.1}$$

La convergencia en q^* con probabilidad $\sim$1 está garantizada siempre que todos los pares estado-acción sigan actualizándose y la tasa de aprendizaje cumpla las condiciones de aproximación estocástica. La política de comportamiento solo determina qué estados se visitan, pero no influye en la convergencia.

SARSA (On-Policy TD)

SARSA es un método de control TD on-policy. El nombre proviene del quinteto $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$ utilizado en la actualización:

$$\begin{aligned} Q(S_t, A_t) &\leftarrow Q(S_t, A_t) \\ &\quad + \alpha\big[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\big] \end{aligned} \tag{2.2}$$

A diferencia del Q-Learning, el valor de bootstrap $Q(S_{t+1}, A_{t+1})$ utiliza la acción A_{t+1} realmente elegida por la política ε-greedy (incluida la exploración). SARSA converge en q^* siempre que todos los estados-acciones se visiten infinitamente y la política converge en el límite hacia la política codiciosa.

Monte Carlo On-Policy

Los métodos MC estiman $q_\pi(s, a)$ como la media de los rendimientos observados para cada par (s, a). La actualización solo se realiza al final del episodio, calculando los rendimientos retrospectivamente. Se utiliza una política ε-soft para garantizar la exploración.

Monte Carlo Off-Policy

La variante off-policy utiliza el muestreo por importancia para estimar q_π a partir de episodios generados por una política de comportamiento $b \neq \pi$. Se utiliza el muestreo por importancia ponderado, que tiene una varianza significativamente menor que el muestreo por importancia ordinario, evitando el problema de la varianza potencialmente infinita en los pesos acumulativos.

2.2. Evaluación / Experimentos

2.2.1. Configuración experimental

Entorno.
`SimpleGrid-8x8-v0`(Figura 2.1) del paquete `gym-simplegrid`: cuadrícula de 8×8 con 64 estados, inicio en el estado 0 y objetivo en el estado 63. Acciones: 4 (arriba, abajo, izquierda, derecha);
Límite: 100 pasos/episodio.

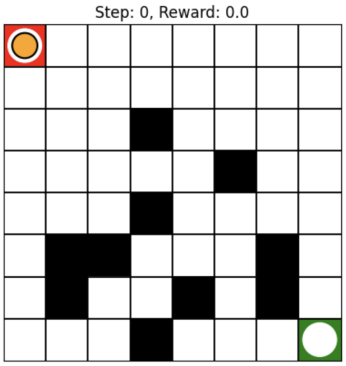

Figura 2.1: Entorno utilizado.

Hiperparámetros. Todos los agentes comparten:

$$\gamma = 0{,}99, \quad \varepsilon_0 = 1{,}0, \quad \varepsilon_{\text{decay}} = 0{,}995$$
$$\varepsilon_{\text{mín}} = 0{,}01, \quad \alpha = 0{,}1 \tag{2.3}$$

Decadencia: $\varepsilon_{t+1} = \text{máx}(\varepsilon_{\text{mín}}, \varepsilon_{\text{decay}} \cdot \varepsilon_t)$. Episodios: 500.

2.2.2. Métricas utilizadas.

Las métricas adoptadas son: recompensa media (últimos 100 ep.), desviación estándar, recompensa máxima, cobertura del espacio de estados (estados con $Q \neq 0$ sobre 64) y longitud de la trayectoria codiciosa comparada con el óptimo de Manhattan (14 pasos).

2.2.3. Resultados obtenidos.

Convergencia.

La Figura 2.2 muestra la media móvil (100 ep.) de la recompensa durante el entrenamiento. Las bandas representan el intervalo de variabilidad.

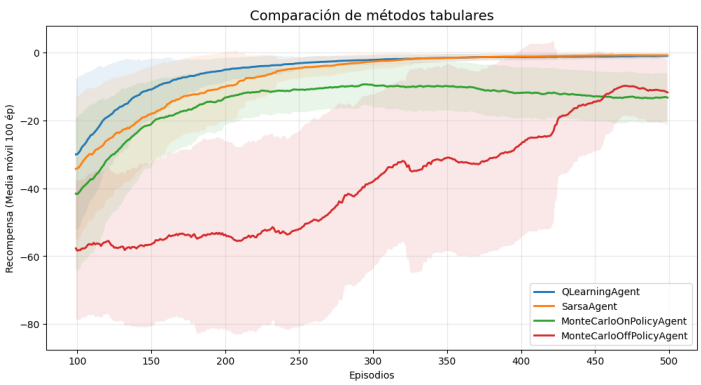

Figura 2.2: Comparación de la convergencia de los cuatro métodos tabulares en `SimpleGrid-8x8-v0`.

La Tabla 2.1 resume el rendimiento al final de los 500 episodios.

Tabla 2.1: Rendimiento de los algoritmos tabulares (500 ep.).

Algoritmo	**Mejor**	**Media$_{100}$**	**Desv. est.**
Q-Learning	−0,3	−0,97	0.78
SARSA	−0,3	−0,81	0.71
MC On-Policy	−0,3	−13,27	6.96
MC Off-Policy	−0,5	−11,74	9.61

Cobertura de los estados.

La Tabla 2.2 muestra el número de estados con valores de Q no nulos.

Tabla 2.2: Cobertura de los estados en la tabla Q.

Algoritmo	**Conocidos**	**%**
Q-Learning	53	82.8
SARSA	53	82.8
MC On-Policy	53	82.8
MC Off-Policy	52	81.3

Trayectorias obtenidas.

Al finalizar el entrenamiento, cada agente se ejecutó con $\varepsilon = 0$. La Tabla 2.3 muestra la longitud del recorrido resultante.

Tabla 2.3: Trayectorias codiciosas tras el entrenamiento.

Algoritmo	**Longitud**	**¿Óptima?**
Q-Learning	14	Sí
SARSA	14	Sí
MC On-Policy	64	+50 pasos
MC Off-Policy	64	+50 pasos

2.2.4. Análisis

Análisis de convergencia. Como podemos ver en la fig. 2.2 y en la tab. 2.1, los métodos TD resultan superiores a los de Monte Carlo. La razón fundamental radica en el mecanismo de bootstrapping: el Q-Learning y el SARSA actualizan $Q(s, a)$ en cada paso temporal, utilizando la estimación actual del estado siguiente como recompensa futura. MC, por el contrario, debe esperar a que finalice el episodio para calcular G_t retrospectivamente. En nuestro entorno, un episodio puede durar hasta 100 pasos sin alcanzar el objetivo: esto genera retornos muy negativos que, al acumularse, producen medias de $-13,27$ (MC On) y $-11,74$ (MC Off). Los métodos TD, al propagar la información paso a paso, permiten al agente llegar al objetivo mucho antes de que el retorno completo esté disponible, como lo confirma la convergencia alrededor del episodio 250 visible en la Figura 2.2. La comparación basada en la tabla 2.1 muestra que SARSA (0,81) obtiene una recompensa media superior a la de Q-Learning (0,97) y una mayor estabilidad. En este escenario determinista, estos resultados se deben probablemente a la estructura de los dos algoritmos; de hecho, el Q-Learning es un método off-policy que actualiza las estimaciones asumiendo siempre la elección de la acción óptima futura (Ec. 2.1), ignorando de hecho el coste de las maniobras exploratorias. Por el contrario, SARSA es un método on-policy que actualiza Q basándose en la

acción ϵ-greedy efectivamente ejecutada (Eq. 2.2). Esto hace que SARSA sea más prudente durante el entrenamiento, aprendiendo a tener en cuenta la penalización derivada de los pasos aleatorios debidos a la exploración, lo que da como resultado un rendimiento medio mejor durante el entrenamiento. Sin embargo, en la prueba final, ambos algoritmos demuestran haber aprendido correctamente la política óptima, completando el recorrido en el número mínimo de 14 pasos (Tab. 2.3). Posteriormente, podemos observar en la Tab. 2.3 que ambos métodos MC producen trayectorias de 64 pasos, es decir, que abarcan la totalidad de las celdas de la cuadrícula de 8×8, lo que indica que el agente está atrapado en un bucle infinito que recorre todos los estados sin llegar nunca al objetivo. En el caso de MC On-Policy, probablemente con solo 500 episodios y actualizaciones solo al final del episodio, el algoritmo no tiene tiempo suficiente para hacer converger Q hacia q_π, especialmente para los estados alejados del objetivo que reciben recompensas extremadamente negativas. Para MC Off-Policy, el problema radica en la varianza del muestreo de importancia: los pesos acumulativos pueden adoptar valores extremos o anularse, lo que hace que episodios enteros sean inutilizables para el aprendizaje. La mejor recompensa de $-0,5$ para MC Off-Policy ($-0,3$ de los demás) es una confirmación más de esta ineficiencia.

Análisis de las tablas Q. Las figuras 2.3, 2.4, 2.5 y 2.6 confirman visualmente el análisis numérico. Los mapas de calor de Q-Learning y SARSA muestran un gradiente cromático coherente: la intensidad del color aumenta hacia la esquina inferior derecha (objetivo), y las flechas de la política greedy convergen uniformemente hacia ella, lo que pone de manifiesto una estimación Q que refleja correctamente la estructura del problema. Por el contrario, las tablas Q de los métodos MC presentan amplias zonas con valores prácticamente nulos y flechas en direcciones incoherentes, lo que indica que, tras 500 episodios, las estimaciones Q aún no son informativas para la mayor parte del espacio de estados. Como podemos ver en la Tab. 2.2, todos los algoritmos cubren aproximadamente el 80 % de la cuadrícula, con una diferencia mínima entre MC Off-Policy (52 estados) y los demás (53). Este dato es significativo, ya que demuestra que el fracaso de los métodos MC no se debe a una exploración insuficiente, sino a la calidad de las estimaciones Q obtenidas en esos 500 episodios. La información recopilada es la misma, pero lo que cambia es la eficiencia con la que se utiliza para actualizar las estimaciones.

2.3. Conclusiones

Los métodos TD obtuvieron mejores resultados que los de MC, gracias al bootstrapping, ya que convergieron rápidamente y generaron trayectorias óptimas (14 pasos). Los métodos MC no lograron aprender una política funcional en 500 episodios, generando trayectorias con bucles (64 pasos), no por falta de exploración, sino por insuficiencia de episodios.

Sin embargo, hay que señalar que el análisis se realiza en un único entorno determinista con 64 estados y pasos limitados, y que los métodos MC podrían beneficiarse de un entrenamiento significativamente más largo, y serían potencialmente más competitivos en entornos estocásticos donde la estimación del retorno completo G_t capta mejor las dependencias a largo plazo.

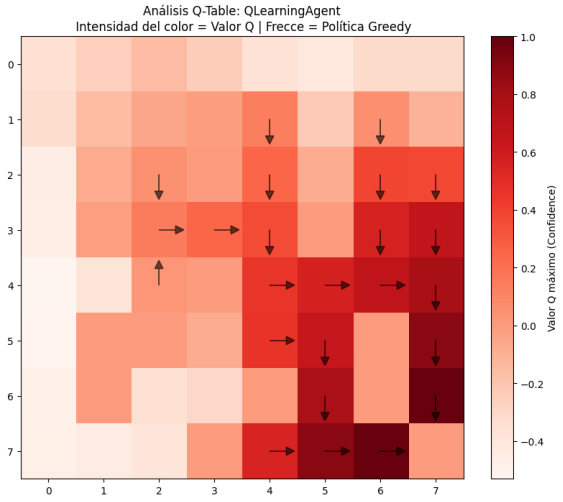

Figura 2.3: Q-table di Q-learning

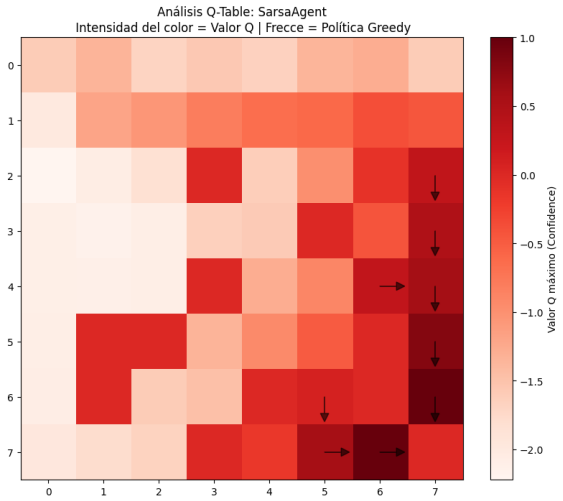

Figura 2.4: Q-table di Sarsa

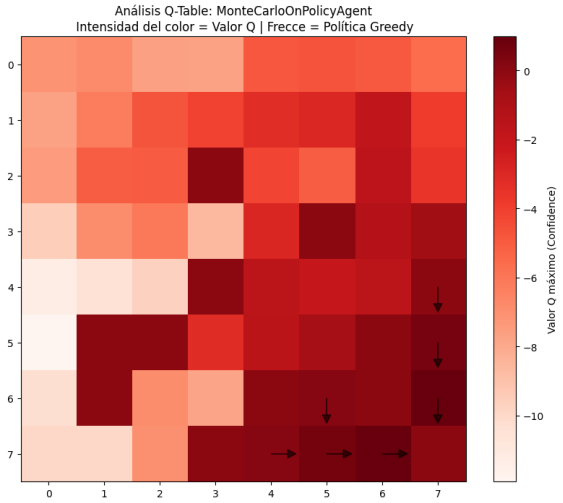

Figura 2.5: Q-table di MC On-policy

2.4. Métodos Aproximados.

Cuando el espacio de estados es continuo o de alta dimensión, los métodos tabulares resultan inaplicables: no es posible al-

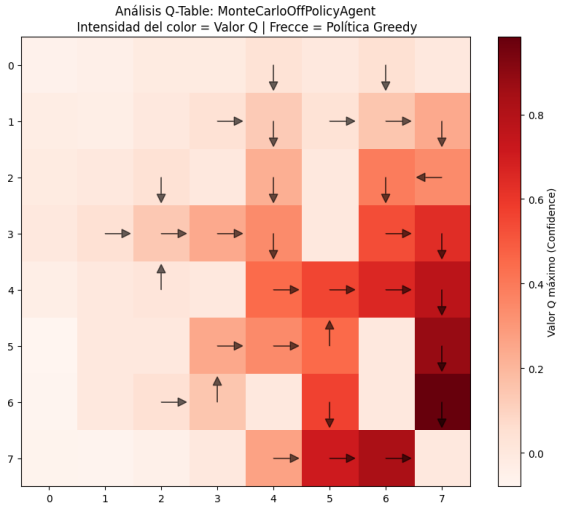

Figura 2.6: Q-table di MC Off-policy

macenar un valor $Q(s,a)$ para cada par estado-acción. Los métodos aproximados sustituyen la tabla por un aproximador paramétrico $\hat{Q}(s,a;\boldsymbol{\theta})$, en este caso una red neuronal, capaz de generalizar a estados nunca visitados.

2.4.1. Objetivos

1. Implementar y comparar SARSA Semi-Gradiente y Deep Q-Network (DQN).

2. Evaluar el impacto de los mecanismos de estabilización (reproducción de experiencias, red objetivo).

3. Realizar estudios de sensibilidad sobre los principales hiperparámetros.

2.4.2. Desarollo.

- **Definición formal:** Para entornos complejos como `FlappyBird-v0`, el problema se reformula como un problema de *aproximación de funciones*, en el que se busca aprender los pesos θ de una red neuronal tal que

$$Q(s,a;\theta) \approx Q^*(s,a).$$

- **Métodos utilizados:** El trabajo se centra en dos algoritmos principales: Deep Q-Network y SARSA Semi-Gradiente.

2.4.3. Algoritmos.

SARSA Semi-Gradient.

SARSA Semi-Gradiente amplía el método tabular SARSA al caso con aproximación de función, en el que la tabla $Q(s,a)$ se sustituye por un aproximador paramétrico $\hat{q}(s,a,\mathbf{w})$, donde $\mathbf{w} \in \mathbb{R}^d$ es el vector de pesos, y actualiza los pesos mediante descenso de gradiente estocástico.

La actualización general para la predicción del valor de acción es:

$$\mathbf{w}_{t+1} \doteq \mathbf{w}_t + \alpha\big[U_t - \hat{q}(S_t,A_t,\mathbf{w}_t)\big]\nabla\hat{q}(S_t,A_t,\mathbf{w}_t) \tag{2.4}$$

tomando ejemplos de la forma $S_t, A_t \to U_t$. En el caso del Sarsa de semigradiente episódico de un paso (on-policy), el objetivo es:

$$U_t = R_{t+1} + \gamma\,\hat{q}(S_{t+1},A_{t+1},\mathbf{w}_t) \tag{2.5}$$

El método se denomina «Semi-gradiente» porque el gradiente se calcula solo con respecto a la estimación $\hat{q}(S_t,A_t,\mathbf{w}_t)$ y no con respecto al objetivo U_t, que también depende de $\mathbf{w}_t$ pero se trata como una constante.

Deep Q-Network.

El Deep Q-Learning (DQN) es una extensión del Q-Learning clásico que utiliza redes neuronales profundas para aproximar la función valor-acción; concretamente, la función $\hat{q}(s,a;\ mathbfw)$ está parametrizada por una red neuronal con pesos $\mathbf{w}$, que recibe como entrada un estado s (un vector numérico o una imagen) y produce como salida un valor $\hat{q}(s,a;\mathbf{w})$ para cada acción posible a.

A diferencia de SARSA, el Q-Learning es un método «off-policy», por lo que el objetivo no utiliza la acción realmente ejecutada por la política de comportamiento, sino el máximo de los valores de todas las acciones en el nuevo estado. En el caso episódico de semigradiente de un paso, el objetivo (Q-Target) es:

$$U_t(\mathbf{w}_t) \doteq R_{t+1} + \gamma \max_{a'} \hat{q}(S_{t+1},a',\mathbf{w}_t) \tag{2.6}$$

DQN introduce dos mecanismos clave de estabilización:

- **Experience Replay**: las transiciones (s_t,a_t,r_t,s_{t+1}) se almacenan en un búfer circular $\mathcal{D}$ de capacidad N. En cada paso de actualización, se muestrea aleatoriamente un minilote de transiciones de $\mathcal{D}$. Esto rompe las correlaciones temporales entre muestras consecutivas y permite reutilizar cada transición en varias actualizaciones, mejorando la eficiencia de muestreo.
- **Target Network:** se utiliza una copia separada de la red, con pesos $\mathbf{w}_{\text{target}}$, para calcular los objetivos. Estos pesos se actualizan (copiados de la red principal) solo cada C pasos, evitando el fenómeno de «persecución del propio objetivo», en el que la red actualiza simultáneamente tanto la función como su objetivo.

Arquitectura de la red.

Ambos algoritmos comparten la misma arquitectura feedforward. El bloque *feature layer* está compuesto por dos capas totalmente conectadas de 128 neuronas cada una con activación ReLU, e incluye una capa de `LayerNorm` después de la primera capa lineal para estabilizar las activaciones y acelerar la convergencia. La capa de salida produce 2 valores (correspondientes a las acciones *noop* y *salta*). La inicialización de los pesos sigue el método ortogonal con ganancia calibrada para ReLU, mientras que los sesgos se inicializan a cero.

2.5. Evaluación / Experimentos

2.5.1. Configuración experimental.

Entorno. `FlappyBird-v0`: el agente controla a un pájaro a través de tubos, recibiendo $+0,1$ por fotograma de supervi-

vencia y $-1,0$ por colisión.
Acciones: 2 (discreto).

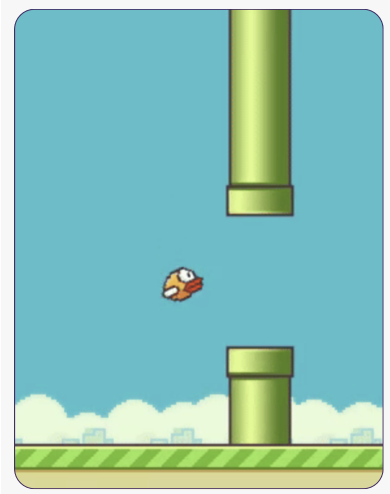

Figura 2.7: Entorno Flappy Bird.

Para acelerar el aprendizaje de los agentes, la función de recompensa original se ha complementado con una lógica avanzada de *Reward Shaping*. Además del bonus de supervivencia por fotograma, se ha introducido una recompensa dinámica basada en la distancia vertical entre el pajarito y un punto objetivo óptimo, situado 20 píxeles por encima del borde del tubo inferior de la siguiente abertura.

Dicho bonus se calcula como:

$$R_{\text{dist}} = \text{máx}\left(0,\ 0{,}5 - \frac{d_v}{150}\right)$$

donde d_v representa la distancia vertical entre el agente y el punto objetivo. Esta formulación incentiva al agente no solo a sobrevivir, sino también a mantener una trayectoria centrada con respecto a los obstáculos. Por último, para reflejar la gravedad del fallo en esta nueva escala de valores, la penalización por colisión se ha incrementado de $-1{,}0$ a $-100{,}0$, proporcionando un gradiente de aprendizaje más marcado para la evitación de choques.

Hiperparámetros. $\gamma = 0{,}99$, decaimiento lineal de ε de 1.0 a 0.01 en el 90 % de los episodios, 10 000 episodios. Parámetros específicos en la Tabla 2.4.

Tabla 2.4: Hiperparámetros específicos.

Parám.	DQN	SARSA S-G
α	10^{-4}	10^{-4}
Batch Size	128	—
Target Update Frequency (C)	1000	—
Memory size (M)	100k	—

2.5.2. Métricas utilizadas.

Recompensa media (últimos 100 episodios), desviación estándar, recompensa máxima, episodio de convergencia (umbral de recompensa > 5), pérdida MSE y prueba final greedy (50 episodios, $\varepsilon = 0$).

2.5.3. Resultados obtenidos.

Convergence and Loss Analysis

La figura 2.8 muestra las tendencias de la pérdida MSE (panel superior, escala logarítmica) y la recompensa media móvil a lo largo de 100 episodios (panel inferior, con intervalos de confianza y un umbral de éxito de 5).

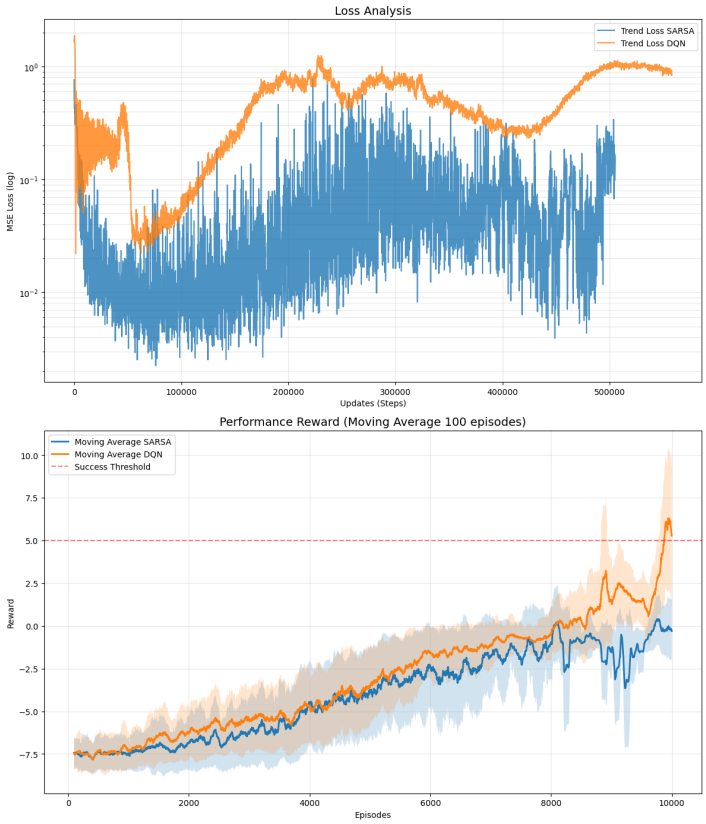

Figura 2.8: Arriba: Pérdida MSE (escala logarítmica) en función de los pasos de actualización. Abajo: recompensa media móvil (100 episodios) con bandas de variabilidad. La línea discontinua roja indica el umbral de éxito (recompensa = 5).

La tabla 2.5 resume el rendimiento cuantitativo al final del entrenamiento.

Tabla 2.5: Rendimiento en `FlappyBird-v0` (10 000 ep.).

Alg.	Mejor	M_{100}	Est.	Conv.
DQN	27,9	2,55	2,30	No
SARSA S-G	4,9	$-2,14$	1,35	No

Prueba final.

Tras el entrenamiento, 50 episodios con $\varepsilon = 0$ (Tabla 2.6).

Tabla 2.6: Prueba final (50 episodios, $\varepsilon = 0$).

Alg.	Media	Desv. est.	Máx.	Mín.
DQN	2,95	5,11	22,5	$-1,9$
SARSA	$-0,09$	2,26	4,6	$-3,3$

Estudios de sensibilidad de hiperparámetros.

Para comprender mejor el comportamiento de los dos algoritmos, se realizaron estudios paramétricos variando un hiperparámetro cada vez, mientras se mantenían los demás fijos en sus valores de referencia (Tabla 2.4).

Target Update Frequency in DQN. La figura 2.9 muestra el efecto de la frecuencia de actualización de la red objetivo ($C \in \{500, 1000, 2000\}$) sobre la convergencia de DQN.

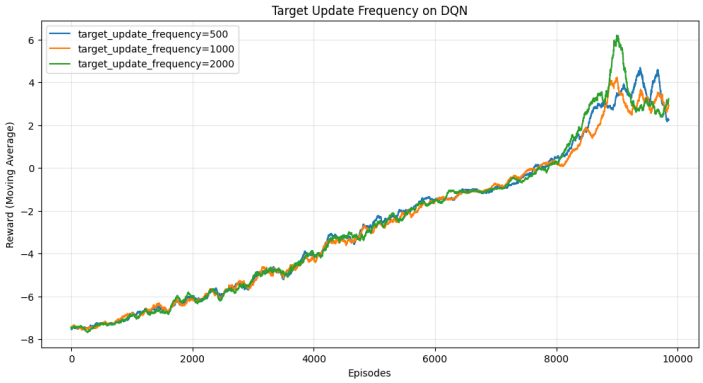

Figura 2.9: Frecuencia de actualización del objetivo en DQN.

Learning rate in DQN. La Figura 2.10 muestra el efecto de Learning rate ($\alpha \in \{10^{-5}, 10^{-4}, 10^{-3}\}$) en DQN.

Figura 2.10: Learning rate en DQN.

Learning rate in SARSA Semi-Gradiente. La Figura 2.11 muestra el mismo estudio sobre SARSA Semi-Gradiente.

Experience Replay en DQN. La Figura 2.12 muestra el efecto del Experience Replay ($M \in \{50\text{k}, 100\text{k}, 200\text{k}\}$) en DQN.

2.5.4. Análisis.

Convergencia de la recompensa. Del gráfico inferior de la Figura 2.8 se desprende que ambos algoritmos parten de valores similares ($\sim -7,5$) en los primeros 1000 episodios, fase

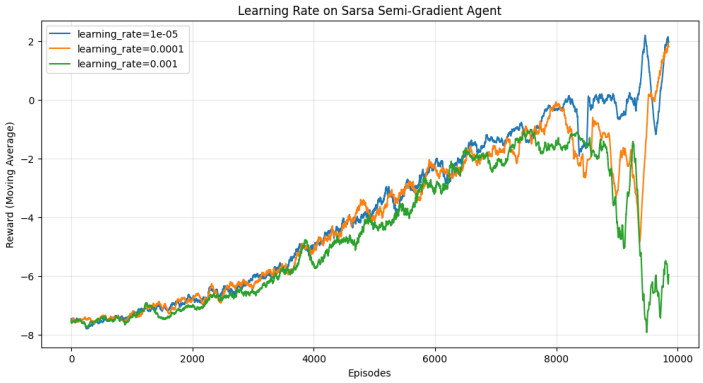

Figura 2.11: Learning rate en SARSA Semi-Gradiente.

Figura 2.12: Tamaño del Experience Replay en DQN.

dominada por la exploración ($\varepsilon > 0,89$). La diferencia entre DQN y SARSA Semi-Gradiente se manifiesta a partir del episodio 3000 y se amplifica progresivamente, en la que DQN alcanza recompensas positivas alrededor del episodio 8000, superando repetidamente el umbral de éxito (recompensa > 5) en la fase final, mientras que SARSA obtiene valores negativos durante toda la duración del entrenamiento. Las bandas de variabilidad muestran que DQN presenta oscilaciones crecientes en la fase final (alta varianza), mientras que SARSA mantiene una variabilidad reducida pero constantemente por debajo de cero. Ninguno de los dos algoritmos alcanza una convergencia estable en 10 000 episodios (Tab. 2.5). En el caso de DQN, las oscilaciones en la fase final de la Figura 2.8 sugieren que el agente sigue explorando regiones del espacio de políticas, con ε ya en su mínimo ($0,01$): un entrenamiento más prolongado podría consolidar los avances. Para SARSA, el problema es más estructural: la ausencia de un búfer de repetición impide un uso eficiente de las muestras en un entorno en el que los episodios útiles solo están disponibles en las fases avanzadas del entrenamiento, cuando el agente ya es poco exploratorio.

Análisis de la loss. El panel superior de la figura 2.8 muestra tendencias muy diferentes: la pérdida de SARSA (azul) parte de valores muy bajos ($\sim 10^{-3}$) y oscila en más de dos órdenes de magnitud (10^{-3}–10^{0}), con picos repentinos que indican inestabilidad en la optimización. La pérdida de DQN (naranja) parte de valores más altos, pero muestra una tendencia general al alza, estabilizándose en torno a 10^{0} en la

segunda mitad del entrenamiento. El hecho de que la recompensa siga mejorando (panel inferior) confirma que el aumento de la pérdida no indica un empeoramiento del aprendizaje, sino una mayor riqueza de la experiencia recopilada.

Prueba final. Los resultados de la prueba final (Tab. 2.6) confirman la ventaja de DQN también en la fase de explotación pura. DQN obtiene una recompensa media positiva de $2,95$, con un máximo de $22,5$ que indica la capacidad de atravesar numerosos tubos, y un mínimo de $-1,9$ que revela fallos limitados. SARSA Semi-Gradiente presenta una media cercana a cero ($-0,09$), con un máximo de $4,6$ que demuestra que el agente ha aprendido a superar algunos tubos, a diferencia de lo observado durante el entrenamiento. La desviación estándar es significativa para ambos (5,11 para DQN, 2,26 para SARSA), lo que refleja la estocasticidad del entorno (posición variable de los tubos). En comparación con los datos de entrenamiento (media de los últimos 100 episodios de $2,55$ para DQN), la media de la prueba final ($2,95$) es ligeramente superior, lo que sugiere que la eliminación de la exploración residual aporta una mejora marginal. En el caso de SARSA, la mejora es más notable: la media de entrenamiento (-2,14) sube a -0,09 en la prueba final, lo que indica que la política subyacente es mejor de lo que muestran las métricas de entrenamiento.

2.5.5. Análisis de sensibilidad.

Target Update Frequency en DQN. La Figura 2.9 muestra que las tres configuraciones ($C \in \{500, 1000, 2000\}$) producen curvas de aprendizaje que se superponen sustancialmente hasta el episodio 7000, para luego divergir en la fase final. Todas alcanzan recompensas positivas y superan el umbral de recompensa > 5 al menos temporalmente. La configuración $C = 500$ alcanza el pico más alto (~ 6) pero con oscilaciones marcadas, en consonancia con el hecho de que las actualizaciones demasiado frecuentes de la red objetivo reducen la estabilidad de los objetivos. $C = 2000$ muestra la convergencia más lenta pero el comportamiento final más estable. En conjunto, DQN demuestra ser robusto con respecto a este hiperparámetro, lo que sugiere que el factor dominante no es la frecuencia de actualización de la red objetivo, sino su presencia.

Learning Rate en la DQN. La Figura 2.10 muestra un efecto más marcado. $\alpha = 10^{-4}$ produce los mejores resultados, alcanzando recompensas positivas con picos alrededor de $+2$. $\alpha = 10^{-5}$ aprende más lentamente pero se mantiene con recompensas negativas (~ -1) incluso en la fase final; probablemente, el paso de actualización es demasiado pequeño para explotar de forma eficaz el presupuesto de 10 000 episodios. $\alpha = 10^{-3}$ muestra una convergencia inicial comparable, pero colapsa en la fase final, con oscilaciones negativas que sugieren inestabilidad en la optimización; en concreto, pasos demasiado grandes provocan actualizaciones excesivas que desestabilizan la función Q aprendida.

Learning Rate en SARSA Semi-Gradiente. La Figura 2.11 revela un comportamiento similar al del learning rate en la DQN, pero con una diferencia significativa en la fase final: de hecho, $\alpha = 10^{-5}$ alcanza el pico más alto ($\sim +2$), superando a $\alpha = 10^{-4}$ (~ 0). $\alpha = 10^{-3}$, en cambio, se desploma bruscamente hacia -8, lo que indica un colapso de la política. Este resultado es particularmente instructivo: sin experience replay, SARSA Semi-Gradiente es mucho más sensible al learning rate que DQN. Un α demasiado grande desestabiliza la red cuando las transiciones están altamente correlacionadas, mientras que un α pequeño, aunque más lento, acumula actualizaciones estables que producen mejores resultados a largo plazo.

Memory Size en DQN. La Figura 2.12 muestra el efecto del tamaño del Experience Replay. $M = 50$k alcanza el mayor rendimiento (~ 6) con un pico alrededor del episodio 9000, seguido de oscilaciones. $M = 100$k (referencia) y $M = 200$k convergen en valores similares (~ 4) pero con tendencias diferentes; de hecho, el búfer más grande ralentiza el aprendizaje inicial porque las transiciones recientes se diluyen entre un mayor número de transiciones obsoletas. El búfer más pequeño, al contener experiencias más recientes, permite actualizaciones más pertinentes para la política actual, pero a costa de una diversidad muestral reducida que genera las oscilaciones observadas. La determinación del tamaño del búfer implica un compromiso entre la recencia temporal de las muestras y la diversidad estadística de las observaciones que contiene, dos propiedades que influyen directamente en la calidad del proceso de aprendizaje.

2.6. Conclusiones.

En conclusión, DQN ha demostrado un rendimiento superior, alcanzando una recompensa máxima de 27,9 frente a los 4,9 de SARSA. La combinación de Experience Replay y Target Network resulta fundamental para estabilizar el aprendizaje en entornos con aproximadores de funciones no lineales (redes neuronales), donde la correlación de las muestras y la inestabilidad de los objetivos suelen impedir la convergencia de métodos on-policy simples como SARSA. Posteriormente, el análisis de sensibilidad reveló que DQN es notablemente robusto a las variaciones en la frecuencia de actualización de la Target Network. Sin embargo, el tamaño del Experience Replay presenta un compromiso: un búfer más pequeño acelera el aprendizaje inicial utilizando datos recientes, pero introduce una varianza que impide una política final estable. Ambos algoritmos mostraron una elevada sensibilidad al parámetro de *learning rate*, donde, en el caso de SARSA, un aprendizaje más lento ($\alpha = 10^{-5}$) resultó ser más ventajoso a largo plazo, lo que sugiere que la actualización constante con datos altamente correlacionados tiende a desestabilizar la función de valor si los pasos de actualización son demasiado grandes. Además, se observó que el aumento de la pérdida MSE en DQN no está correlacionado con un empeoramiento del rendimiento; al contrario, coincide con la obtención de mayores recompensas. Esto indica que, a medida que el agente sobrevive más tiempo, se enfrenta a experiencias más complejas y diversificadas que ponen a prueba la red neuronal, enriqueciendo el aprendizaje a pesar del aumento inicial del error cuadrático medio. Por último, aunque DQN logró superar puntualmente el umbral de éxito, ninguno de los dos agentes alcanzó una convergencia estable en 10.000 episodios. Esto sugiere que, para un entorno de alta varianza y recompensas dispersas como Flappy Bird, sería necesario un entrenamiento más prolongado o la implementación de mejoras como Double DQN.

Bibliografía

[1] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3), 235–256.

[2] Kuleshov, V., & Precup, D. (2014). Algorithms for multi-armed bandit problems. *arXiv preprint arXiv:1402.6028*.

[3] Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

Listar todas las referencias utilizadas en el trabajo, siguiendo un formato bibliográfico estándar de LaTeX o de alguna asociación (APA, IEEE, etc.). La información obligatoria para que tipo de publicación la puede consultar en https://bibtex.eu/fields/ Se recomienda utilizar un gestor de referencias como BibTeX para facilitar la organización de las citas solo si va a poner mucha bibliografía.